

내 손으로 조작하며 배우는 5일 강화학습

260206 5일차: AI 엘리베이터

Git vs GitHub

Git (깃)

- 내 컴퓨터의 버전 관리 프로그램
- 로컬에서 파일 변경 내역 기록

GitHub (깃허브)

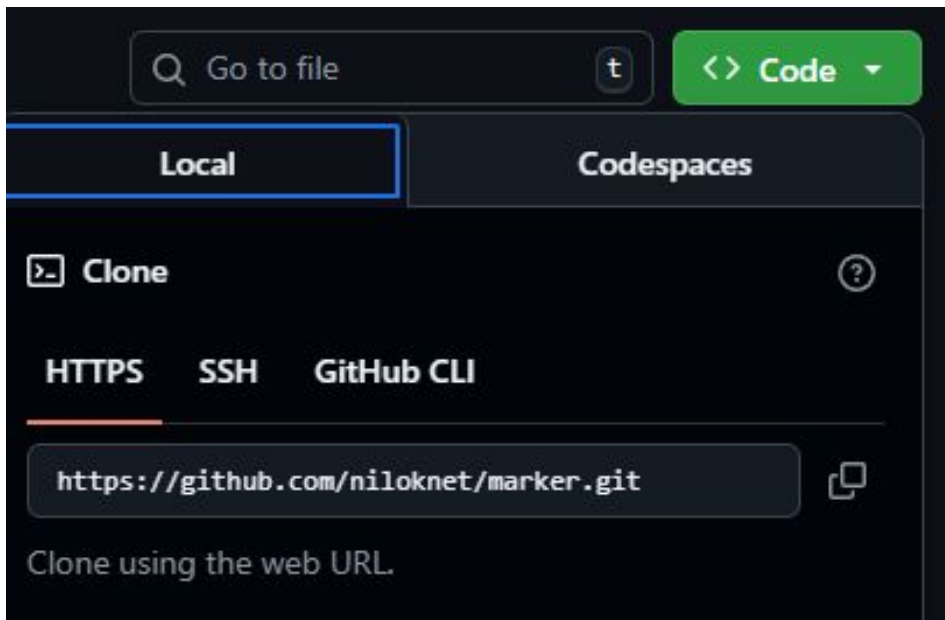
- 클라우드 원격 저장소 서비스
- 협업, 백업, 공유 플랫폼

Windows Git 환경 구성

- <https://git-scm.com/install/windows>
- 기본 옵션으로 설치 진행
- 설치 확인
- 사용자 이름/이메일 설정 (필수)
 - 터미널에서 실행:
 - `git config --global user.name "Your Name"`
 - `git config --global user.email "email@example.com"`

Git을 통한 Github 저장소 복사

```
git clone -b 2026-02 https://github.com/niloknet/marker.git
```



프로젝트 개요

- 엘리베이터 효율성 최적화를 위한 **AI** 에이전트 개발
- 랜덤 에이전트와 **AI** 에이전트 성능 비교
- **Streamlit**, **Gradio**을 활용한 대시보드 구현

BuildingElevator-v0 환경

- 4층 건물, 1대 엘리베이터 커스텀 Gym 환경
- 승객 스폰: 푸아송 분포 ($\text{spawn_rate}=0.02\sim0.03$)
 - 10스텝당 1명

BuildingElevator-v0 환경

- **상태 (State)**

- 엘리베이터: 층, 방향, 문 상태, 탑승 인원
- 호출: 상행/하행 호출, 내부 버튼

- **행동 (Action)**

- 0: STAY (대기)
- 1: MOVE_UP (상승)
- 2: MOVE_DOWN (하강)
- 3: OPEN_DOOR (문 열기)
- 4: CLOSE_DOOR (문 닫기)

BuildingElevator-v0 환경

- **보상 (Reward)**

- 매 스텝 대기 패널티: $-0.05 \times$ 대기 인원 수
- 승객 도착 시: +15.0
- 잘못된 행동 패널티: 문 열린 채 이동 (-10.0)
- 목표: 대기 시간 최소화 + 빠른 승객 운송

- **종료조건 (Truncated)**

- 1000 스텝 (고정 시간내에 최대의 승객 운송)

랜덤 에이전트 테스트

- 베이스라인 성능 측정을 위한 랜덤 에이전트 실행
 - 무작위 행동 선택 (탐색 전략 없음)
 - AI 모델과 비교를 위한 기준 설정

RSL-RL

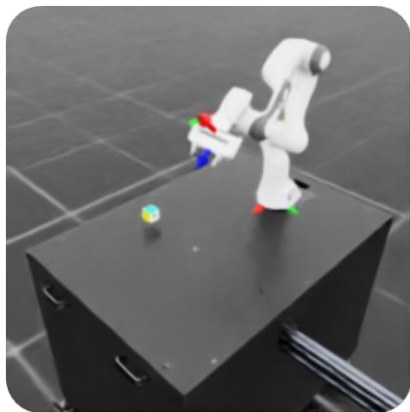
- 취리히 연방 공과대학교의 **RSL**에서 개발한 강화학습 라이브러리
- 로봇이 "어떻게" 움직일지 학습
 - **반사 훈련**: 강건한 보행 정책 학습
 - **맹목적 이동**: 목표는 모르지만 안정적인 이동 방법 학습
 - **GPU 최적화**: 대량 데이터 처리
- RSL-RL은 다양한 **RL** 알고리즘을 지원하여 로봇의 행동 정책을 학습
 - **PPO**: 가장 널리 사용되는 기본 알고리즘
 - **A2C**: 빠른 학습을 위한 동기적 방법
 - **SAC, TD3**: 연속 제어를 위한 **off-policy** 알고리즘

Nvidia Isaac Lab

- **대규모 병렬화:** 단일 GPU에서 수천 개의 로봇을 동시 훈련
- **물리 정확도:** 마찰, 접촉력 등 복잡한 물리 상호작용 시뮬레이션
- **훈련 시간 단축:** 수년에서 수분으로 감소

VLA(Vision-Language-Action)

- 멀티모달 입력: 카메라(Vision) + 텍스트 명령(Language)
- https://youtu.be/59JTCvpG_Ec
- <https://youtu.be/Sq1QZB5baNw>



이미지

태그	내용
지시	"큐브를 집으세요"
로봇 행동	[0.02, -0.01, 0.05, 0.0, 0.1, -0.05, 1.0]



이미지

태그	내용
지시	"큐브를 목표지점으로 가져가세요"
로봇 행동	[-0.01, 0.03, -0.02, 0.1, -0.05, 0.0, 0.0]

프롬프트 작성 팁

- 명확하고 구체적인 맥락 제시: 목적, 요구사항, 산업 용어 포함
- 복잡한 워크플로우를 작은 논리적 단계로 나누기
- 출력 형식 명시: 형식, 구조, 톤, 길이 정의
 -  나쁜 예시: "AI in healthcare에 대해 작성해줘"
 -  좋은 예시: "임상 의료 전문가를 위한 AI 응용 사례에 대한 500단어 기사 작성"

프롬프트 맥락 관리

- **작업별 초기화:** 각 작업에 집중된 맥락만 제공
- **맥락 관리의 균형**
 - ⚠ 너무 적은 맥락: 환각(hallucination) 유발
 - ⚠ 너무 많은 맥락: 토큰 한계 초과 및 초점 상실

LLM과 협업하여 코딩하기

- 테스트 케이스를 먼저 작성
- 한 번에 하나의 작업만 수행 - 주요 기능 전환 시 새 세션 시작
- A/B 테스트와 성능 모니터링
- 깨끗한 입력 값을 검증, 보장해서 신뢰할 수 있는 출력을 얻어야함

감사합니다